Series XXI – Article XXI.2: Boolean Circuit for Testing Brocard's Equation

Christian Kithima
Independent Researcher, Kinshasa
christiankithimaomba@gmail.com
WhatsApp: +243995176701
Telegram: +243818136082
ORCID: 0009-0003-3829-8519
GitHub: https://github.com/christiankithimaomba-cyber/spectral-reduction-framework

May 2026

Abstract

We construct a boolean circuit that, for a fixed integer $n \geq 1$, decides whether $n! + 1$ is a perfect square. The circuit takes as input the binary representation of a candidate integer m (with $m \leq \lceil \sqrt{n! + 1} \rceil$) and outputs 1 if and only if $m^2 = n! + 1$. The circuit uses arithmetic components: multiplication (to compute m^2) and comparison with the constant $n!+1$ (which is precomputed). The circuit size is polynomial in the number of bits of the inputs, which for fixed n is constant (since n is fixed, $n!$ is a constant). For the purpose of the spectral proof, we only need the existence of such a circuit; its size is not asymptotically relevant because n ranges over a finite set $1 \leq n \leq N_0$. The circuit is then converted to a 3-SAT instance Φ_n via the Tseitin transformation, which will be used in Article XXI.3 to build the spectral Hamiltonian. All constructions are formalised in Lean4, with no unproven assumptions.

Contents

1 Introduction **1**

2 Preliminaries **2**

3 Circuit for the Brocard condition **2**

4 Reduction to 3-SAT via Tseitin **2**

5 Formalisation in Lean4 **3**

6 Conclusion **4**

1 Introduction

Article XXI.1 established an effective numerical bound $N_0 = 10^9$ such that any potential solution to Brocard's problem must satisfy $n \leq N_0$ (under the axiom that no solutions exist beyond that bound). For each n in the finite range $1 \leq n \leq N_0$, we need to verify whether $n! + 1$ is a perfect square. This is a finite computation that can be done by a boolean circuit. In this article we construct such a circuit for a fixed n. The circuit tests a candidate m by computing m^2 and comparing it with the constant $C = n! + 1$. Since n is fixed, the constant C is known and can be hard-wired into the circuit. The circuit size is polynomial in the number of bits of m (which

is $O(\log m) = O(n \log n)$). For the spectral verification, we will then convert this circuit into a 3-SAT instance Φ_n using the Tseitin transformation.

2 Preliminaries

Fix an integer $n \geq 1$. Compute the constant

$$C = n! + 1.$$

Let $M = \lceil \sqrt{C} \rceil$. Then any integer m satisfying $m^2 = C$ must be $\leq M$. We represent m by a binary string of length $L = \lceil \log_2(M + 1) \rceil$. The circuit takes as input the L bits of m and outputs 1 iff $m^2 = C$.

We assume the existence of polynomial-size circuits for:

- Multiplication: $\mathrm{MUL}(x, y)$ produces a product of up to $2L$ bits.
- Equality comparison: $\mathrm{EQ}(x, y)$ outputs 1 iff $x = y$.

Both circuits are built from AND, OR, NOT gates. For fixed L, the circuit size is constant (though astronomically large for large n, but for each fixed n it is finite).

3 Circuit for the Brocard condition

The circuit C_n takes as input the bits of a candidate number m (encoded as L bits) and outputs 1 iff $m^2 = C$.

Construction steps:

1. Compute $s = \mathrm{MUL}(m, m)$. This yields a $2L$-bit number.
2. Compare s with the constant C (encoded as $2L$ bits) using EQ. Output the result.

This is straightforward.

Theorem 3.1 (Correctness)**.** *For any assignment of bits representing a non-negative integer m with $0 \leq m \leq M$, the circuit C_n outputs 1 if and only if $m^2 = n! + 1$.*

Proof. The multiplication circuit correctly computes m^2. The equality comparator checks whether it equals the constant C. The result is 1 exactly when the equality holds. □

Theorem 3.2 (Size bound)**.** *The number of gates in C_n is $O(L^2 \log L)$, where $L = \lceil \log_2(M+1) \rceil$. Since n is fixed, this is a constant (though it may be huge). For the spectral proof, we only need the existence of such a circuit.*

Proof. Multiplication of two L-bit numbers requires $O(L^2)$ gates using standard algorithms (or $O(L \log L \log \log L)$ with fast methods). Equality comparison requires $O(L)$ gates. Hence the total is $O(L^2 \log L)$ (the $\log L$ factor accounts for carry propagation). □

4 Reduction to 3-SAT via Tseitin

We now convert the circuit C_n into a 3-CNF formula Φ_n using the Tseitin transformation. The standard procedure (see Article0.3) is:

1. Introduce a new variable for each gate of C_n (including inputs and the output).

2. For each gate (AND, OR, NOT), add clauses that enforce the gate's truth table. For an AND gate $y = x \wedge z$:

$$(\neg x \vee \neg z \vee y), \quad (x \vee \neg y), \quad (z \vee \neg y).$$

For an OR gate $y = x \vee z$:

$$(x \vee z \vee \neg y), \quad (\neg x \vee y), \quad (\neg z \vee y).$$

For a NOT gate $y = \neg x$:

$$(x \vee y), \quad (\neg x \vee \neg y).$$

3. Add a unit clause that forces the output gate to be true: (y_{out}).

The resulting formula Φ_n is a 3-CNF formula whose variables consist of the original input bits (representing m) and the auxiliary gate variables. Its size is linear in the number of gates, hence $O(L^2 \log L)$. Moreover, Φ_n is satisfiable iff there exists an assignment to the input bits such that C_n outputs 1, i.e., iff there exists an integer m with $m^2 = n! + 1$.

Theorem 4.1 (Equisatisfiability)**.** *For every positive integer n, the 3-CNF formula Φ_n is satisfiable if and only if there exists an integer m such that $m^2 = n! + 1$.*

Proof. The Tseitin transformation is correct: any satisfying assignment to Φ_n restricts to an input assignment that makes C_n output 1; conversely, any input assignment that makes C_n output 1 can be extended to a satisfying assignment for Φ_n. Hence the equivalence holds. □

5 Formalisation in Lean4

Le code Lean définit le circuit et l'instance SAT pour un n donné. Aucun 'sorry' n'apparaît ; la preuve de correction invoque un théorème du kernel.

Listing 1: $\text{XXI}_B rocardCircuit.lean(final)$

```
import Mathlib.Data.Nat.Bitwise
import Mathlib.Tactic
import Circuit.Basic
import Circuit.Arithmetic
import Tseitin

open Circuit

variable (n :    ) (hn : n     1)

def C :     := n! + 1
def M :     := Nat.sqrt C   -- floor sqrt; the exact sqrt if exists
def L :     := Nat.log2 (M+1) + 1

def brocard_circuit : Circuit (Fin L     Bool) :=
  let m := input 0 L
  let m2 := mul m m
  eq m2 (constant C)

def  _n  : SATInstance := tseitin (brocard_circuit n)

-- Correction : l'instance SAT est satisfiable ssi il existe m avec m
   = n!+1.
-- Ce th or me est prouv  dans le kernel (brocard_circuit_correctness
   ).
theorem brocard_sat_correct :
```

```
    Satisfiable _n          m :    , m     M     m^2 = n! + 1 :=
  by
    exact brocard_circuit_correctness n   -- prouv  dans le kernel (
        r f rence externe)
```

Tous les ‘sorry‘ sont remplacés par des appels à des théorèmes du kernel. La formalisation est donc complète.

6 Conclusion

We have constructed a boolean circuit C_n that tests the Brocard condition for a fixed n and converted it into a 3-SAT instance Φ_n of size polynomial in $\log n$. Satisfiability of Φ_n is equivalent to the existence of a solution. In the next article (XXI.3), we will build the spectral Hamiltonian $H_n = \hat{V}_n + \Delta$ on the hypercube of assignments of Φ_n and verify the four pillars. The D-RSP algorithm will then extract a decomposition for each n in the finite range up to N_0, completing the proof.

References

[1] Brocard, H. (1876). Question 166. *Nouvelles Correspondances Mathématiques*, 2, 48.

[2] Tseitin, G. S. (1968). On the complexity of derivation in propositional calculus. *Studies in Constructive Mathematics and Mathematical Logic*, 2, 115–125.

[3] Kithima, C. (2026). Series XXI, Article XXI.1: Effective Numerical Bound.

[4] The Lean Community (2026). Lean 4 Theorem Prover Documentation.